SQL Basics: Banking Use Case

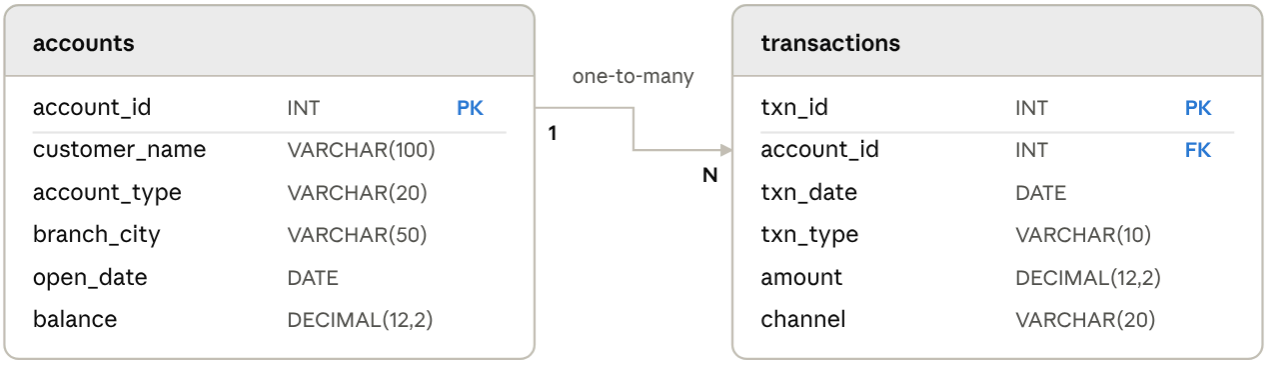
Use case

A retail bank keeps customer accounts in one table and every money movement in another. Each account belongs to one customer, sits in one branch city and is either a Savings or Current account. Every credit or debit on that account is recorded as a transaction, along with its date, amount and channel (UPI, ATM or NEFT).

The analytics team wants to check the quality of this data before reporting on it, and to adjust the table structure as new requirements come in.

Table structure

One account has many transactions



transactions.account_id references accounts.account_id. PK = primary key, FK = foreign key.

ER diagram · 2 tables, 1 relationship

Each account can have zero or many transactions; each transaction belongs to exactly one account.

Sample tables

The data includes a few deliberate issues (NULLs, a duplicate, an account with no transactions) so the profiling questions have something to find.

accounts

account_id	customer_name	account_type	branch_city	open_date	balance (INR)
101	Amit Sharma	SAVINGS	Delhi	2021-03-15	85,000.00
102	Priya Verma	CURRENT	Mumbai	2020-07-01	2,40,000.00

account_id	customer_name	account_type	branch_city	open_date	balance (INR)
103	Rahul Gupta	SAVINGS	Delhi	2022-11-20	12,000.00
104	Sneha Iyer	SAVINGS	Bengaluru	2023-01-05	NULL
105	Karan Mehta	CURRENT	Mumbai	2019-05-10	5,10,000.00
106	Neha Kapoor	NULL	Pune	2024-02-28	3,000.00

transactions

txn_id	account_id	txn_date	txn_type	amount (INR)	channel
1	101	2026-09-01	CREDIT	50,000.00	NEFT
2	101	2026-09-03	DEBIT	12,000.00	UPI
3	102	2026-09-04	DEBIT	75,000.00	NEFT
4	102	2026-09-04	DEBIT	75,000.00	NEFT
5	103	2026-09-05	DEBIT	2,000.00	ATM
6	103	2026-09-07	CREDIT	15,000.00	NULL
7	105	2026-09-08	CREDIT	2,00,000.00	NEFT
8	105	2026-09-10	DEBIT	45,000.00	UPI
9	104	2026-09-12	DEBIT	NULL	UPI
10	101	2026-09-15	DEBIT	18,000.00	UPI

DDL

Run this once to create both tables and load the sample data. Syntax works on MySQL and PostgreSQL.

```
-- Table 1: Customer bank accounts
CREATE TABLE accounts (
    account_id     INT PRIMARY KEY,
    customer_name  VARCHAR(100) NOT NULL,
    account_type   VARCHAR(20),          -- 'SAVINGS' / 'CURRENT'
    branch_city    VARCHAR(50),
    open_date      DATE,
    balance        DECIMAL(12,2)
);

-- Table 2: Transactions on those accounts
CREATE TABLE transactions (
    txn_id      INT PRIMARY KEY,
    account_id  INT,
```

```
    txn_date    DATE,
    txn_type    VARCHAR(10),            -- 'CREDIT' / 'DEBIT'
    amount      DECIMAL(12,2),
    channel     VARCHAR(20),            -- 'UPI' / 'ATM' / 'NEFT'
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

-- Sample data
INSERT INTO accounts VALUES
(101, 'Amit Sharma', 'SAVINGS', 'Delhi',     '2021-03-15', 85000.00),
(102, 'Priya Verma', 'CURRENT', 'Mumbai',    '2020-07-01', 240000.00),
(103, 'Rahul Gupta', 'SAVINGS', 'Delhi',     '2022-11-20', 12000.00),
(104, 'Sneha Iyer',  'SAVINGS', 'Bengaluru', '2023-01-05', NULL),
(105, 'Karan Mehta', 'CURRENT', 'Mumbai',    '2019-05-10', 510000.00),
(106, 'Neha Kapoor', NULL,      'Pune',      '2024-02-28', 3000.00);

INSERT INTO transactions VALUES
(1,  101, '2026-09-01', 'CREDIT', 50000.00,  'NEFT'),
(2,  101, '2026-09-03', 'DEBIT',  12000.00,  'UPI'),
(3,  102, '2026-09-04', 'DEBIT',  75000.00,  'NEFT'),
(4,  102, '2026-09-04', 'DEBIT',  75000.00,  'NEFT'),
(5,  103, '2026-09-05', 'DEBIT',  2000.00,   'ATM'),
(6,  103, '2026-09-07', 'CREDIT', 15000.00,  NULL),
(7,  105, '2026-09-08', 'CREDIT', 200000.00, 'NEFT'),
(8,  105, '2026-09-10', 'DEBIT',  45000.00,  'UPI'),
(9,  104, '2026-09-12', 'DEBIT',  NULL,      'UPI'),
(10, 101, '2026-09-15', 'DEBIT',  18000.00,  'UPI');
```

Segment 1: Clauses and table structure altering

These five questions cover filtering, sorting, grouping, joins and changing a table's structure. Run them in the order given; Q4 and Q5 change the tables.

Q1. WHERE + ORDER BY

Show all UPI debits above ₹10,000 made in September 2026, largest amount first. Return txn_id, account_id, txn_date and amount.

Expected output (3 rows):

txn_id	account_id	txn_date	amount
8	105	2026-09-10	45,000.00
10	101	2026-09-15	18,000.00

txn_id	account_id	txn_date	amount
2	101	2026-09-03	12,000.00

Transaction 9 is a UPI debit but does not appear, because a NULL amount never passes a comparison like `amount > 10000`.

Q2. GROUP BY + HAVING

Which branch cities hold a total balance of more than ₹1,00,000? Show the city, number of accounts and total balance.

Expected output (1 row):

branch_city	num_accounts	total_balance
Mumbai	2	7,50,000.00

Delhi just misses at ₹97,000. Bengaluru's only balance is NULL, so its total is NULL and is filtered out.

Q3. JOIN + CASE + LIMIT

Show the top 3 customers by net flow, where net flow = total credits minus total debits.

Expected output (3 rows):

customer_name	net_flow
Karan Mehta	1,55,000.00
Amit Sharma	20,000.00
Rahul Gupta	13,000.00

Neha Kapoor is absent because an inner join drops accounts with no transactions. Sneha Iyer's net flow is NULL; on PostgreSQL a descending sort puts NULL first, so the query needs `NULLS LAST` or `COALESCE` to get this result.

Q4. ALTER TABLE: rename and modify a column

Rename `branch_city` to `branch_location` in the accounts table, and increase `customer_name` from 100 to 150 characters.

Expected output: the statements run without error and the table structure becomes:

Column	Type	Key / constraint
account_id	INT	PK
customer_name	VARCHAR(150)	NOT NULL
account_type	VARCHAR(20)	
branch_location	VARCHAR(50)	
open_date	DATE	
balance	DECIMAL(12,2)	

All 6 rows and their values stay the same; only the column definitions change.

Q5. ALTER TABLE: add a column and a constraint

Add an `email` column (VARCHAR(100)) to accounts. Then add a CHECK constraint so that a transaction amount must be greater than zero.

Expected output:

- accounts now has 7 columns, and `email` is NULL for all 6 existing rows.
- The CHECK constraint is added successfully, since every existing non-NULL amount is positive. The NULL amount on transaction 9 does not violate it, because CHECK only rejects values that evaluate to false.
- Test: inserting a new transaction with amount -500 is rejected with a check constraint violation error.

Segment 2: Data profiling

What is data profiling? Data profiling is the process of examining a dataset to understand its structure, content and quality before using it. It answers questions like how many rows there are, which values are missing, what range the numbers fall in, whether there are duplicates, and whether records link correctly across tables. In finance this matters because a NULL amount or a duplicate debit directly distorts balances and reports.

These questions run on the tables after Segment 1, so the city column is now `branch_location`.

Q6. Row counts and distinct values

How many rows are in each table? How many distinct account types and distinct branch locations exist?

Expected output:

Measure	Value
Rows in accounts	6
Rows in transactions	10
Distinct account types	2
Distinct branch locations	4

The distinct account type count is 2, not 3, because `COUNT(DISTINCT ...)` ignores the NULL on account 106.

Q7. Missing values

Count the NULL values in the key columns of both tables: account_type and balance in accounts, and txn_date, amount and channel in transactions.

Expected output:

Table	Column	NULL count
accounts	account_type	1
accounts	balance	1
transactions	txn_date	0
transactions	amount	1
transactions	channel	1

Q8. Summary statistics

For each transaction type, show the number of transactions and the minimum, maximum, average and total amount.

Expected output (2 rows):

txn_type	txn_count	min_amount	max_amount	avg_amount	total_amount
CREDIT	3	15,000.00	2,00,000.00	88,333.33	2,65,000.00
DEBIT	7	2,000.00	75,000.00	37,833.33	2,27,000.00

`COUNT(*)` gives 7 debits, but MIN, MAX, AVG and SUM use only the 6 non-NULL amounts, so the debit average is ₹2,27,000 / 6.

Q9. Duplicate detection

Find possible duplicate transactions: rows with the same account, date, type and amount.

Expected output (1 row):

account_id	txn_date	txn_type	amount	occurrences
102	2026-09-04	DEBIT	75,000.00	2

Transactions 3 and 4 are identical apart from txn_id, which suggests a double posting worth checking.

Q10. Accounts with no transactions

List the accounts that have no transactions at all.

Expected output (1 row):

account_id	customer_name
106	Neha Kapoor

This needs a LEFT JOIN from accounts to transactions; an inner join would hide the account entirely.